

You are my personal Java instructor. Your goal is to teach me Java from absolute beginner level to advanced level Java Programming .

Teaching Rules

1. Assume I know NOTHING about programming.
2. Teach module-by-module.
3. Never jump ahead.
4. Move to the next topic ONLY when I type:

Show more

MODULE 8: Exception Handling

Welcome back. Since you've completed Modules 0-7, you already know classes, objects, inheritance, and the basics of how Java programs run. Now we tackle something every real-world Java program needs: handling things that go wrong.

Let's start with the foundational question.

What is an Exception?

An exception is an **event that disrupts the normal flow of a program's execution**, usually because something unexpected happened — dividing by zero, accessing an array index that doesn't exist, trying to read a file that isn't there, and so on.

Without exception handling, your program would just **crash** the moment something goes wrong. Exception handling lets you **anticipate problems and respond gracefully** instead of letting the whole program die.

Step 3: Syntax

Here is the basic syntax for handling an exception:

```
java

try {
    int result = 10 / 0;
    System.out.println(result);
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero!");
}
```

```
} finally {  
    System.out.println("This always runs.");  
}
```

Step 4: Breakdown (line by line, symbol by symbol)

Let's break this down piece by piece — nothing skipped.

Line 1: `try {`

Symbol	Meaning
--------	---------

<code>try</code>	A keyword that marks the start of a block of code that <i>might</i> throw an exception.
------------------	--

<code>{</code>	Opens the block — everything between this <code>{</code> and its matching <code>}</code> is "watched" for errors.
----------------	---

Line 2: `int result = 10 / 0;`

Symbol	Meaning
--------	---------

<code>int</code>	Declares a variable of type integer (you know this from Module 1).
------------------	--

<code>result</code>	The variable name.
---------------------	--------------------

<code>=</code>	Assignment operator.
----------------	----------------------

<code>10 / 0</code>	Integer division by zero — this is mathematically undefined, and in Java, this specific operation throws an exception at runtime.
---------------------	--

<code>;</code>	Statement terminator (every Java statement ends with this).
----------------	---

Line 3: `System.out.println(result);`

This line is **never reached** if line 2 throws an exception. Execution jumps immediately to the `catch` block. This is critical to understand: **once an exception is thrown, Java abandons the rest of the `try` block instantly** and looks for a matching `catch`.

Line 4: `} catch (ArithmeticException e) {`

Symbol	Meaning
--------	---------

<code>}</code>	Closes the <code>try</code> block.
----------------	------------------------------------

Symbol	Meaning
<code>catch</code>	A keyword that defines a block to handle a specific type of exception.
<code>(ArithmeticException e)</code>	This says: "If the exception thrown is of type <code>ArithmeticException</code> (or a subclass of it), catch it and store a reference to it in a variable named <code>e</code> ."
<code>ArithmeticException</code>	A class built into Java (part of <code>java.lang</code>) representing arithmetic errors like division by zero.
<code>e</code>	A variable name (you could call it anything — <code>e</code> is just convention, short for "exception").
<code>{</code>	Opens the catch block.

Line 5: `System.out.println("Cannot divide by zero!");`

This runs **only if** the exception was caught. It's your "Plan B" — what to do instead of crashing.

Line 7: `} finally {`

Symbol	Meaning
<code>finally</code>	A keyword marking a block that runs no matter what — whether an exception occurred or not, whether it was caught or not.

Line 8: `System.out.println("This always runs.");`

This prints **regardless** of whether the division failed. `finally` is typically used for cleanup — closing files, closing database connections, releasing resources — things that *must* happen no matter what.

Step 5: Keyword Explanation (First-Time Keywords)

`try`

- **What it means:** Marks a block of "risky" code that the JVM should monitor for exceptions.
- **Why it exists:** Without it, Java wouldn't know which part of your code you're trying to "protect." It also signals to the compiler: "I know this might fail, and I'm handling it."

- **When to use it:** Wrap any code that might fail at runtime — file access, network calls, division, array access, parsing user input, etc.

catch

- **What it means:** Defines what to do *if* a specific exception occurs in the preceding `try` block.
- **Why it exists:** It lets you separate "normal logic" from "error-handling logic," and respond differently depending on *what kind* of error happened.
- **When to use it:** Always pair with `try`. You can have multiple `catch` blocks for different exception types.

finally

- **What it means:** Code that executes after `try`/`catch`, no matter what happened — success, failure, or even if you `return` from inside the try block.
- **Why it exists:** Some actions (like closing a file) must happen regardless of success or failure. Without `finally`, you'd have to duplicate cleanup code in every possible exit path.
- **When to use it:** Resource cleanup — closing files, database connections, network sockets.

ArithmeticException

- **What it means:** A built-in Java class representing an error in an arithmetic operation.
- **Why it exists:** Java needs a structured way to represent *what kind* of problem occurred, so your code can react specifically.
- **When to use it:** You don't "use" it directly — Java *throws* it automatically when arithmetic fails (e.g., integer division by zero). You write a `catch` block for it when you anticipate this might happen.

Step 6: Compiler Behavior

Here's something important: **the Java compiler does NOT know in advance that `10 / 0` will fail.** This is a **runtime** error, not a compile-time error.

- At **compile time**, the compiler only checks: is the syntax valid? Does `ArithmeticException` exist? Are types correct?
- The compiler does **not** execute `10 / 0` — it just compiles it into bytecode instructions for division.
- The error only happens when the JVM actually *executes* that division during the program's run.

This is different from **checked exceptions** (which we'll get to shortly), where the compiler *forces* you to handle them even before running the program.

Step 7: Runtime Behavior

When the program actually runs:

1. The JVM executes `10 / 0`.
 2. The JVM detects this is an illegal operation.
 3. The JVM creates a new **object** of type `ArithmeticException` (yes — exceptions are objects!).
 4. This object is "**thrown**" — meaning normal execution stops immediately.
 5. The JVM looks for a matching `catch` block. It finds `catch (ArithmeticException e)` — a match.
 6. The exception object is assigned to `e`.
 7. The catch block executes.
 8. The `finally` block executes.
 9. Execution continues normally after the try-catch-finally structure.
-

Step 8: Memory Behavior

- When the exception occurs, a new object (the exception) is created on the **heap** (just like any other object in Java).
 - This object carries information: a message, the exception type, and a **stack trace** (a record of exactly which method calls led to the error).
 - Once the `catch` block finishes and `e` goes out of scope, that exception object becomes eligible for **garbage collection**, just like any unused object.
 - The `try` block itself doesn't allocate special memory — it's a control-flow structure, not a data structure.
-

Step 9: Common Mistakes

1. **Catching the wrong exception type**

```
java
```

```
try {  
    int result = 10 / 0;  
}
```

```
} catch (NullPointerException e) { // Wrong type – won't catch ArithmeticExce
    System.out.println("This will NOT run");
}
```

This will crash because `ArithmeticException` doesn't match `NullPointerException`. The program terminates with an **uncaught exception**.

2. Putting `finally` to skip cleanup believing catch is "enough"

Beginners often forget `finally` runs even if you `return` from inside `try`. This causes confusing bugs when cleanup logic seems to "not be needed" but actually is.

3. Empty catch blocks (silent failure)

```
java
catch (ArithmeticException e) {
    // nothing here – beginner mistake!
}
```

This hides bugs. Always at least log or print something.

4. Believing `try` prevents the error from happening

It doesn't prevent it — it only lets you *respond* to it after it happens.

That's the foundation of exception handling. Type `next` when you're ready to move to `throw` and `throws`, or let me know if you want me to clarify anything from this section first.

next

MODULE 8 (continued): `throw` and `throws`

Now that you understand `try`, `catch`, and `finally`, let's look at how exceptions actually get *created* and *passed up* — using `throw` and `throws`.

These two look similar but do **completely different jobs**. Beginners confuse them constantly, so pay close attention to the distinction.

- `throw` → **actually throws** an exception object, right now, at this line.

- `throws` → **declares** that a method *might* throw an exception, so the caller knows to handle it.
-

Step 3: Syntax

```
java

public class BankAccount {

    public void withdraw(double balance, double amount) throws InsufficientFunds
        if (amount > balance) {
            throw new InsufficientFundsException("Not enough balance!");
        }
        System.out.println("Withdrawal successful");
    }
}
```

And calling it:

```
java

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        try {
            account.withdraw(1000, 5000);
        } catch (InsufficientFundsException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

(Don't worry — `InsufficientFundsException` is a *custom* exception. We'll build it properly in the next section. For now, focus on `throw` and `throws`.)

Step 4: Breakdown

```
public void withdraw(double balance, double amount) throws
InsufficientFundsException {
```

Symbol	Meaning
<code>public</code>	Access modifier — you learned this in Module 7. Method is accessible from anywhere.
<code>void</code>	Return type — this method returns nothing.
<code>withdraw</code>	Method name.
<code>(double balance, double amount)</code>	Parameters — two <code>double</code> values passed in.
<code>throws</code>	Keyword — declares that this method <i>might</i> throw the named exception, and does NOT handle it internally. It passes the responsibility to whoever calls this method.
<code>InsufficientFundsException</code>	The type of exception this method might throw.
<code>{</code>	Opens the method body.

```
if (amount > balance) {
```

Standard `if` condition from Module 2 — checks whether the withdrawal amount exceeds the balance.

```
throw new InsufficientFundsException("Not enough balance!");
```

Symbol	Meaning
<code>throw</code>	Keyword — actually creates and throws the exception <i>right now</i> . Execution of this method stops immediately at this line.
<code>new</code>	Keyword you know from Module 7 — creates a new object on the heap.
<code>InsufficientFundsException(...)</code>	Calls the constructor of this exception class.
<code>"Not enough balance!"</code>	A <code>String</code> literal passed into the constructor — this becomes the exception's message .
<code>;</code>	Statement terminator.

The moment `throw` executes, **everything after it in this method is skipped**. The exception propagates upward — out of `withdraw()` — looking for a `catch` block

somewhere in the call chain.

In `main`:

```
java
try {
    account.withdraw(1000, 5000);
} catch (InsufficientFundsException e) {
    System.out.println("Error: " + e.getMessage());
}
```

Symbol	Meaning
<code>account.withdraw(1000, 5000)</code>	Calls the method — <code>1000</code> becomes <code>balance</code> , <code>5000</code> becomes <code>amount</code> .
<code>e.getMessage()</code>	A method (inherited from the <code>Exception</code> class) that retrieves the message you passed into the constructor — <code>"Not enough balance!"</code> .
<code>+</code>	The <code>String</code> concatenation operator (from Module 1) — joins <code>"Error: "</code> with the message.

Step 5: Keyword Explanation

`throws`

- **What it means:** A declaration in a method's signature saying "executing this method might result in this type of exception being thrown — I'm not handling it myself."
- **Why it exists:** Java wants **compile-time safety** for certain exceptions (called *checked exceptions* — explained below). If a method might throw a checked exception, the compiler forces you to either catch it or declare `throws` so the caller is warned.
- **When to use it:** When writing a method that throws a **checked exception**, and you've decided *not* to handle it inside that method, but instead let the caller deal with it.

`throw`

- **What it means:** An actual command that creates an exception object and immediately disrupts normal execution.

- **Why it exists:** This is *how* exceptions come into existence in the first place. Someone, somewhere, has to call `throw` (either you, or internally inside Java's own code, like the JVM throwing `ArithmeticException` for you).
- **When to use it:** When your code detects a condition that should NOT be allowed to continue normally — invalid input, broken business rules, impossible states.

Step 6: Checked vs Unchecked Exceptions (Critical Concept)

This is a concept every Java developer must understand deeply.

Java has **two categories** of exceptions:

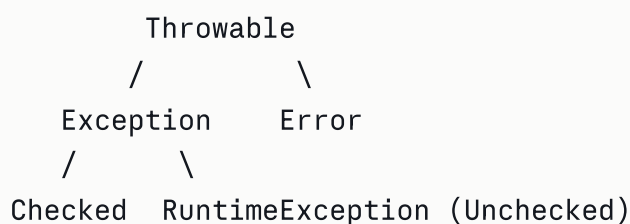
1. Checked Exceptions

- These are exceptions the **compiler forces you to handle**.
- Examples: `IOException`, `SQLException`, your custom `InsufficientFundsException` (if it extends `Exception`).
- If a method can throw one, you **must** either:
 - Catch it with `try-catch`, OR
 - Declare it with `throws` and push the responsibility to the caller.
- If you do neither, **the code won't even compile**.

2. Unchecked Exceptions

- These are exceptions the compiler does **NOT** force you to handle.
- Examples: `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`.
- These typically represent **programming bugs** rather than recoverable conditions — Java doesn't force you to catch them because, in theory, you should fix the bug rather than handle it.
- They extend `RuntimeException`.

This is exactly why our earlier example with `10 / 0` didn't need a `throws` declaration — `ArithmeticException` is unchecked.



Step 7: Compiler Behavior

- If you write a method that calls code which throws a **checked** exception, and you do NOT catch it or declare `throws`, **the compiler will refuse to compile your program**. This is a compile-time error, not a runtime one.
- If you forget `throws` on `withdraw()` in our example (assuming `InsufficientFundsException extends Exception`), you'd get a compiler error like:

```
error: unreported exception InsufficientFundsException; must be caught or declared to be thrown
```

- For **unchecked** exceptions, the compiler says nothing — it compiles fine even if you never catch them, and the risk only shows up at runtime.

Step 8: Runtime Behavior

1. `main()` calls `account.withdraw(1000, 5000)`.
2. Inside `withdraw`, the condition `5000 > 1000` is true.
3. `throw new InsufficientFundsException(...)` executes — an exception object is created on the heap.
4. Execution of `withdraw()` halts immediately — `System.out.println("Withdrawal successful")` never runs.
5. Java looks for a `catch` block **in the calling method** (`main`), because `withdraw` itself has no `try-catch` — it just declared `throws`.
6. `main`'s `try-catch` catches it, and the `catch` block executes.

This upward search is called **exception propagation** — if a method doesn't catch its own exception, Java keeps walking up the **call stack** until it finds a `catch` that matches, or until it reaches `main()` and crashes the program entirely.

Step 9: Memory Behavior

- The exception object lives on the **heap**, like any object.
- It carries a **stack trace** — a snapshot of the **call stack** at the exact moment `throw` executed. This is why error messages show you the full chain: which method called which method, down to the exact line.
- As the exception propagates upward, each method's **stack frame** is "popped" off the call stack (since those methods are abandoned mid-execution) until a matching `catch` is found.

- Once caught and the variable `e` goes out of scope, the exception object becomes eligible for garbage collection.

Step 10: Common Mistakes

1. Forgetting `throws` for checked exceptions

java

```
public void withdraw(double balance, double amount) { // missing throws!  
    throw new InsufficientFundsException("Not enough balance!");  
}
```

This won't compile if `InsufficientFundsException` is checked.

2. Confusing `throw` with `throws`

java

```
public void withdraw() throw InsufficientFundsException { } // WRONG syntax
```

`throws` (with an "s") goes in the method signature. `throw` (no "s") goes inside the method body to actually fire the exception.

3. Throwing exceptions but never catching them anywhere

If no method in the entire call chain catches it, the program **terminates abruptly** with a printed stack trace.

4. Using exceptions for normal control flow

Beginners sometimes use exceptions like an `if-else` replacement. This is bad practice — exceptions are for *exceptional*, unexpected situations, not routine logic. It also hurts performance, since creating exception objects is relatively expensive (they capture the entire stack trace).

5. Catching `Exception` (the generic parent) instead of the specific type

java

```
catch (Exception e) { } // too broad - hides real bug info
```

This technically works (since `InsufficientFundsException` is an `Exception`), but it's sloppy — you lose the ability to respond differently to different problems.

Type `next` when ready to move to **custom exceptions** — where we'll build `InsufficientFundsException` from scratch and understand exactly how to design your own exception classes.

next

MODULE 8 (continued): Custom Exceptions

So far we've used `ArithmeticException` (built into Java) and referenced `InsufficientFundsException` without actually building it. Now let's build our own exception class from scratch and understand exactly how it works.

Why does Java let you create your own exceptions?

Java's built-in exceptions are generic — `ArithmeticException`, `NullPointerException`, etc. But your *business logic* often has problems that Java has no name for. "Insufficient funds" is not a concept Java knows about — it's specific to *your* banking application. Custom exceptions let you give meaningful, specific names to problems unique to your program.

Step 3: Syntax

```
java

public class InsufficientFundsException extends Exception {

    public InsufficientFundsException(String message) {
        super(message);
    }
}
```

And using it:

```
java

public class BankAccount {

    private double balance;

    public BankAccount(double balance) {
        this.balance = balance;
    }
}
```

```

    }

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException("Cannot withdraw " + amount + "
        }
        balance -= amount;
        System.out.println("Withdrawal successful. New balance: " + balance);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(1000);
        try {
            account.withdraw(5000);
        } catch (InsufficientFundsException e) {
            System.out.println("Transaction failed: " + e.getMessage());
        }
        System.out.println("Program continues running normally.");
    }
}

```

Step 4: Breakdown

```
public class InsufficientFundsException extends Exception {
```

Symbol	Meaning
<code>public class</code> <code>InsufficientFundsException</code>	Declares a new class (you know this from Module 7) named <code>InsufficientFundsException</code> .
<code>extends</code>	Keyword you've seen in Module 7 (inheritance) — here it means our class inherits from <code>Exception</code> .
<code>Exception</code>	Java's built-in class representing a checked exception. By extending it, our class becomes an exception type itself.

This single line is what makes `InsufficientFundsException` a real, throwable exception. Without `extends Exception` (or `extends RuntimeException`, or some other `Throwable` subclass), Java would refuse to let you `throw` it.

```
public InsufficientFundsException(String message) {
```

This is a **constructor** (Module 7 concept) — it runs when you do `new`
`InsufficientFundsException(...)`.

Symbol	Meaning
<code>public</code>	Accessible from anywhere.
<code>InsufficientFundsException(String message)</code>	Constructor name (must match class name) taking one parameter — a <code>String</code> called <code>message</code> .

`super(message);`

Symbol	Meaning
<code>super</code>	Keyword — refers to the parent class (<code>Exception</code>). <code>super(...)</code> calls the parent's constructor.
<code>(message)</code>	Passes our <code>message</code> string up to <code>Exception</code> 's own constructor.

This is critical: `Exception` already has built-in machinery to **store** a message and make it retrievable later via `getMessage()`. By calling `super(message)`, we're reusing that machinery instead of reinventing it.

Step 5: Keyword Explanation

`extends` (recap, applied to exceptions)

- **What it means:** Establishes an inheritance relationship — `InsufficientFundsException` is a `Exception`.
- **Why it exists:** So you can create specialized exception types while still getting all the built-in behavior (`getMessage()`, stack traces, compatibility with `catch (Exception e)`, etc.) for free.
- **When to use it:** Always, when creating a custom exception — extend either `Exception` (checked) or `RuntimeException` (unchecked), depending on whether you want the compiler to force handling.

`super`

- **What it means:** A reference to the immediate parent class. `super(...)` specifically calls the parent's constructor.
- **Why it exists:** When a child class is constructed, the parent part of the object must also be initialized. Java requires the parent's constructor to run — `super(...)` lets

you control *what* gets passed to it.

- **When to use it:** Whenever your constructor needs to initialize inherited state — here, the exception's internal message field, which lives in `Exception` / `Throwable`.

Step 6: Checked or Unchecked? Your Choice.

Notice we wrote:

```
java
```

```
public class InsufficientFundsException extends Exception
```

This makes it a **checked** exception — which is *why* `withdraw()` needed `throws InsufficientFundsException` in its signature, and why `main()` was *forced* by the compiler to wrap the call in `try-catch`.

If instead we wrote:

```
java
```

```
public class InsufficientFundsException extends RuntimeException
```

It would become **unchecked** — the compiler would no longer force anyone to catch it. The choice depends on intent:

Choose <code>Exception</code> (checked) when...	Choose <code>RuntimeException</code> (unchecked) when...
---	--

The caller <i>should</i> be forced to think about this failure	The error usually indicates a programming bug
--	---

The condition is recoverable and expected sometimes	You want lighter-weight code without mandatory try-catch everywhere
---	---

Example: insufficient funds, file not found	Example: invalid argument passed by a careless programmer
---	---

Step 7: Compiler Behavior

- The compiler checks: does `InsufficientFundsException` properly extend a class that ultimately derives from `Throwable`? If not — error.
- Because we chose `extends Exception`, the compiler treats it as **checked**, and enforces the catch-or-declare rule everywhere it's used.
- The compiler verifies `super(message)` matches an existing constructor in `Exception` that accepts a `String`. (It does — `Exception(String message)` is a real

constructor inherited all the way from `Throwable`.)

Step 8: Runtime Behavior

Walking through `main()`:

1. `new BankAccount(1000)` — creates a `BankAccount` object on the heap with `balance = 1000`.
2. `account.withdraw(5000)` is called inside the `try` block.
3. Inside `withdraw`, `5000 > 1000` is `true`.
4. `new InsufficientFundsException("Cannot withdraw 5000.0. Balance is only 1000.0")` is executed:
 - A new exception object is created on the heap.
 - Its constructor runs, which calls `super(message)`, storing the message inside the inherited `Throwable` fields.
 - At this moment, Java also automatically captures the **current call stack** into this object.
5. `throw` fires — `withdraw()` halts immediately. `balance -= amount` and the `println` **never execute**.
6. The exception propagates up to `main()`.
7. `main()`'s `catch (InsufficientFundsException e)` matches — it catches it.
8. `e.getMessage()` retrieves the stored string.
9. `"Transaction failed: " + e.getMessage()` is printed.
10. Execution continues normally — `"Program continues running normally."` prints.

This last point matters: **catching an exception "heals" the program**. Once caught, Java doesn't keep crashing — your program resumes normal execution right after the catch block.

Step 9: Memory Behavior

- The `InsufficientFundsException` object lives on the **heap**, like all objects.
- It internally stores (inherited from `Throwable`):
 - The message string (a reference to a `String` object, also on the heap).
 - A `stackTrace` array — internally captured at the moment of construction, describing every method call that led here.
- When `e` goes out of scope after the catch block finishes, this entire object becomes eligible for garbage collection.

- Each abandoned method (`withdraw`'s stack frame) is removed from the **call stack** the moment `throw` propagates past it.

Step 10: Common Mistakes

1. Forgetting `super(message)`

java

```
public InsufficientFundsException(String message) {  
    // forgot super(message)  
}
```

This compiles fine, but `e.getMessage()` will return `null` later, because the message was never actually stored.

2. Extending the wrong class

java

```
public class InsufficientFundsException extends RuntimeException // if you me
```

This silently changes behavior — the compiler stops forcing callers to handle it, which might let real bugs slip through unnoticed in production.

3. Overly generic exception names

Beginners write things like `MyException` or `Error1`. Always name exceptions to describe *what went wrong* — `InsufficientFundsException`, `InvalidAgeException`, `UserNotFoundException`.

4. Putting business logic inside the exception class

Exception classes should generally just carry information (a message, maybe extra fields like the attempted amount). Don't put unrelated logic or calculations there.

5. Throwing exceptions for trivial validation that could be handled with simple `if` logic and a return value

Not every problem needs an exception. Reserve them for genuinely *exceptional* situations.

Quick Recap of Module 8 So Far

Keyword	Purpose
<code>try</code>	Wraps risky code
<code>catch</code>	Handles a specific exception type
<code>finally</code>	Always runs, used for cleanup
<code>throw</code>	Actually fires an exception, right now
<code>throws</code>	Declares a method might throw a checked exception
<code>extends Exception</code>	Creates a checked custom exception
<code>extends RuntimeException</code>	Creates an unchecked custom exception
<code>super(...)</code>	Calls the parent constructor

Type `next` to move on to **multi-catch blocks, exception hierarchy with multiple catches, and the try-with-resources statement** — the final advanced pieces of exception handling before we move to Module 9 (Collections Framework).

[next](#)

MODULE 8 (continued): Multi-Catch, Catch Ordering, and Try-With-Resources

This is the final piece of exception handling before we move to Collections. We'll cover three things:

1. Handling **multiple exception types** for one `try` block.
2. The **ordering rule** for catch blocks (a common source of compiler errors).
3. **Try-with-resources** — a cleaner way to handle resources like files, replacing manual `finally` cleanup.

Part A: Multiple Catch Blocks

Step 3: Syntax

```
java
```

```

public class Main {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30};

        try {
            int index = 5;
            System.out.println(numbers[index] / 0);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid array index: " + e.getMessage());
        } catch (ArithmeticException e) {
            System.out.println("Math error: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Some other problem occurred: " + e.getMessage())
        }
    }
}

```

Step 4: Breakdown

Symbol	Meaning
<code>int[] numbers = {10, 20, 30};</code>	Array declaration from Module 5 — an array of 3 integers.
<code>numbers[index]</code>	Accessing index <code>5</code> , but the array only has indices <code>0</code> , <code>1</code> , <code>2</code> . This is out of bounds .
<code>catch (ArrayIndexOutOfBoundsException e)</code>	Catches specifically the "bad index" problem.
<code>catch (ArithmeticException e)</code>	Catches division-by-zero type problems (wouldn't actually run here, since the first catch fires first).
<code>catch (Exception e)</code>	A catch-all — catches <i>anything</i> else that is an <code>Exception</code> or subclass of it.

In this example, `numbers[index]` fails **before** Java even attempts the `/ 0` part, so `ArrayIndexOutOfBoundsException` is thrown, and the **first matching catch block runs** — Java checks catch blocks **top to bottom**, and stops at the first match. The other two catch blocks are skipped entirely.

Step 5: Keyword Explanation

There's no brand-new keyword here, but there is a brand-new **rule** worth treating like one: **catch block ordering**.

The Ordering Rule

- **What it means:** When you have multiple `catch` blocks, Java checks them **in the order you wrote them**, top to bottom, and uses the **first one that matches**.
- **Why it exists:** Exception classes form a hierarchy (remember the diagram from earlier: `Throwable` → `Exception` → `RuntimeException` → `ArithmeticException`, etc.). If you put a **general** catch (like `Exception`) *before* a **specific** one (like `ArithmeticException`), the general one will *always* match first, making the specific one **unreachable** — and Java's compiler will actually flag this as an **error**.

Example of the mistake:

```
java
try {
    int result = 10 / 0;
} catch (Exception e) {           // too general, placed first
    System.out.println("General error");
} catch (ArithmeticException e) { // UNREACHABLE - compiler error!
    System.out.println("Math error");
}
```

This **fails to compile** with an error like:

```
error: exception ArithmeticException has already been caught
```

Rule of thumb: always order catch blocks from **most specific** → **most general**.

Part B: Multi-Catch with `|` (Pipe Operator)

Sometimes you want to handle two *different* exception types **the same way**, without duplicating code.

Step 3: Syntax

```
java
try {
    int[] numbers = {1, 2, 3};
    System.out.println(numbers[10] / 0);
} catch (ArrayIndexOutOfBoundsException | ArithmeticException e) {
```

```
System.out.println("Either an index problem or a math problem: " + e.getMessage());
}
```

Step 4: Breakdown

Symbol	Meaning
<code>ArrayIndexOutOfBoundsException ArithmeticException</code>	Multi-catch syntax — this single catch block matches <i>either</i> exception type.
<code> </code>	The pipe operator — here used not as "bitwise OR" (a different meaning you might see elsewhere) but specifically as a separator between alternative exception types in a catch clause.

Step 5: Keyword/Symbol Explanation

`|` in multi-catch context

- **What it means:** "Catch this block if the exception is of *this* type OR *that* type."
- **Why it exists:** Without it, you'd need two separate catch blocks with **uplicated** code inside, which violates good coding practice (repeating yourself).
- **When to use it:** When two or more exception types should be handled **identically**.

Important restriction

The exception types joined by `|` must **not** be related by inheritance (one can't be a subclass of the other) — Java requires them to be independent branches of the hierarchy, otherwise it's redundant and the compiler rejects it.

Part C: Try-With-Resources

This is one of the most useful modern Java features. It automatically closes resources (files, database connections, etc.) for you — no need to manually write `finally { resource.close(); }`.

Step 3: Syntax

```
java
import java.io.FileReader;
import java.io.IOException;
```

```

public class Main {
    public static void main(String[] args) {
        try (FileReader reader = new FileReader("data.txt")) {
            int character;
            while ((character = reader.read()) != -1) {
                System.out.print((char) character);
            }
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}

```

Step 4: Breakdown

Symbol	Meaning
<code>import java.io.FileReader;</code>	Keyword <code>import</code> — brings in a class from Java's library so we can use it without writing its full path every time. We'll formally cover packages in Module 14, but for now: <code>import</code> just means "I want to use this tool from Java's toolbox."
<code>import java.io.IOException;</code>	Imports the exception class associated with input/output failures.
<code>try (FileReader reader = new FileReader("data.txt"))</code>	This is try-with-resources : the parentheses right after <code>try</code> declare a resource that Java will automatically close when the try block finishes — whether it finished successfully or via an exception.
<code>FileReader reader = new FileReader("data.txt")</code>	Creates an object that can read characters from a file named <code>data.txt</code> .
<code>int character;</code>	Declares a variable to hold one character at a time (as an <code>int</code> , because <code>read()</code> returns <code>-1</code> at end-of-file, which can't be represented as a <code>char</code>).
<code>while ((character = reader.read()) != -1)</code>	Reads one character at a time; <code>-1</code> signals "no more characters" (end of file).
<code>(char) character</code>	A typecast (Module 1 concept) — converts the <code>int</code> value back into a readable <code>char</code> .
<code>catch (IOException e)</code>	Catches any input/output failure — e.g., file not found.

Step 5: Keyword Explanation

import

- **What it means:** Tells the compiler "I want to use this class, which lives in this package (a folder-like grouping of related classes)."
- **Why it exists:** Java's standard library has *thousands* of classes. Without `import`, you'd have to write the full path every single time (e.g., `java.io.FileReader` instead of just `FileReader`).
- **When to use it:** Any time you use a class that isn't in `java.lang` (the only package automatically available without importing) or in your own current file/package.

Try-with-resources mechanism

- **What it means:** Any object used in `try (...)` must implement a special interface called `AutoCloseable` (or `Closeable`). Java automatically calls `.close()` on it once the try block ends — success or failure.
- **Why it exists:** Before this feature (introduced in Java 7), developers had to manually write:

```
java

FileReader reader = null;
try {
    reader = new FileReader("data.txt");
    // use reader
} finally {
    if (reader != null) {
        reader.close();
    }
}
```

This was verbose and error-prone — people forgot the null check, forgot to close, or closed in the wrong order with multiple resources. Try-with-resources eliminates all of that boilerplate.

- **When to use it:** Anytime you work with files, network sockets, database connections — anything that holds an external resource that must be released.

Step 6: Compiler Behavior

- For try-with-resources, the compiler checks that `FileReader` implements `AutoCloseable`. If it doesn't, compilation fails.

- For multi-catch (`|`), the compiler verifies the listed exception types are not subclasses of one another.
 - For catch ordering, the compiler statically analyzes the exception hierarchy and rejects unreachable catch blocks.
-

Step 7: Runtime Behavior

For try-with-resources specifically, here's the *exact* order of operations:

1. `FileReader` is created and opened.
2. The code inside the `try` block runs (reading characters).
3. **Regardless of whether an exception occurred**, Java automatically calls `reader.close()` — this happens *before* any `catch` block executes.
4. *Then*, if an exception occurred, the matching `catch` block runs.

This guarantees the file handle is released immediately, even if something goes wrong mid-read — you don't have to remember to do it yourself.

Step 8: Memory Behavior

- `FileReader` doesn't just use heap memory for the Java object — it also holds a reference to an **operating system-level file handle**, a limited resource outside the JVM's normal garbage-collected memory.
 - This is *exactly why* try-with-resources matters: garbage collection eventually cleans up the Java object, but **it does NOT promptly close the underlying OS file handle**. If you don't explicitly close it, you can run out of available file handles in a long-running program even though "memory" looks fine.
 - `close()` releases that OS-level resource immediately and deterministically, rather than waiting on the garbage collector's schedule (which is unpredictable).
-

Step 9: Common Mistakes

1. **Wrong catch order (most common compiler error in this topic)**

```
java
```

```
catch (Exception e) { }  
catch (ArithmeticException e) { } // compiler error: unreachable
```

2. **Using multi-catch with related exception types**

java

```
catch (Exception | ArithmeticException e) { } // ERROR: ArithmeticException i
```

3. Forgetting that `e` in multi-catch is effectively final

You cannot reassign `e` to a new value inside a multi-catch block — it's implicitly treated as a constant reference.

4. Manually closing resources instead of using try-with-resources, then forgetting to close on the exception path

java

```
FileReader reader = new FileReader("data.txt");  
// do stuff  
reader.close(); // never reached if an exception happens above this line!
```

5. Assuming try-with-resources catches the exception for you

It does **not** — it only guarantees `close()` is called. You still need a `catch` block (or `throws`) to actually handle the exception.

Module 8 Complete — Full Summary

Concept	Keyword/Syntax
Risky code block	<code>try { }</code>
Handle specific exception	<code>catch (Type e) { }</code>
Always-runs cleanup	<code>finally { }</code>
Fire an exception now	<code>throw new SomeException(...)</code>
Declare possible exception	<code>methodName() throws SomeException</code>
Custom checked exception	<code>extends Exception</code>
Custom unchecked exception	<code>extends RuntimeException</code>
Handle multiple types, same logic	<code>catch (TypeA TypeB e)</code>
Auto-close resources	<code>try (Resource r = new Resource())</code>

Type `next` to begin **Module 9: Collections Framework**, starting with `ArrayList`.

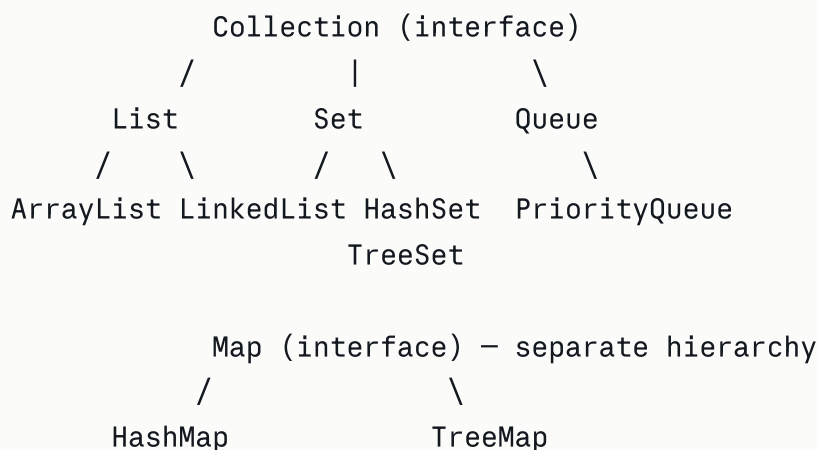
MODULE 9: Collections Framework

Before touching any specific class, you need to understand **why this entire module exists**.

In Module 5, you learned **arrays**. Arrays have a serious limitation: **their size is fixed once created**. If you create an array of size 5, it can *never* grow to hold a 6th element — you'd have to create an entirely new, bigger array and copy everything over manually.

The **Collections Framework** is Java's solution: a set of pre-built, ready-to-use data structures that can **grow and shrink dynamically**, and come with built-in methods for sorting, searching, removing, and more — so you don't have to write that logic yourself every time.

The Big Picture (Don't skip this — it's the map for the whole module)



- **List** → ordered, allows duplicates, accessed by index (like a flexible array).
- **Set** → no duplicates allowed.
- **Queue** → designed for processing items in a particular order (first-in-first-out, priority, etc.).
- **Map** → stores **key-value pairs** (not a **Collection** itself, but part of the same framework).

We'll go through each one, in the order listed in your roadmap, starting with **ArrayList**.

ArrayList

Step 3: Syntax

```
java

import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();

        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");

        System.out.println(names);
        System.out.println(names.get(1));

        names.remove("Bob");
        System.out.println(names);
        System.out.println(names.size());
    }
}
```

Output:

```
[Alice, Bob, Charlie]
Bob
[Alice, Charlie]
2
```

Step 4: Breakdown

```
import java.util.ArrayList;
```

`ArrayList` lives in the `java.util` package — that's where almost all of the Collections Framework lives. You'll see `java.util` repeatedly throughout this module.

```
ArrayList<String> names = new ArrayList<>();
```

This line has several pieces you haven't seen before. Let's go symbol by symbol.

Symbol	Meaning
<code>ArrayList</code>	The class — a dynamic, resizable list implementation.
<code><String></code>	Generics syntax — tells Java "this <code>ArrayList</code> will only hold <code>String</code> objects." We'll deeply cover generics in Module 10, but you need a basic working understanding now.
<code>names</code>	Variable name — a reference variable.
<code>=</code>	Assignment.
<code>new</code> <code>ArrayList<>()</code>	Creates a new, empty <code>ArrayList</code> object on the heap.
<code><></code>	The diamond operator — an empty pair of angle brackets. It tells Java "infer the type from the left side" (i.e., it already knows it's <code><String></code> because of the declared type), so you don't have to repeat <code><String></code> twice.
<code>;</code>	Statement terminator.

```
names.add("Alice");
```

Symbol	Meaning
<code>names</code>	The <code>ArrayList</code> object reference.
<code>.add(...)</code>	A method belonging to <code>ArrayList</code> — inserts an element at the end of the list.
<code>"Alice"</code>	A <code>String</code> literal — the value being added.

```
System.out.println(names);
```

When you print a collection directly, Java automatically calls its **internal** `toString()` **method**, which formats it as `[element1, element2, element3]`. You didn't write this formatting — `ArrayList` provides it for you.

```
names.get(1);
```

Symbol	Meaning
<code>.get(int index)</code>	Retrieves the element at the specified index.

Symbol	Meaning
<code>1</code>	Index <code>1</code> — remember, indexing starts at <code>0</code> (Module 5 concept), so this retrieves the second element, <code>"Bob"</code> .

`names.remove("Bob");`

Symbol	Meaning
<code>.remove(Object o)</code>	Removes the first occurrence of the specified value (not an index — this is the <i>overloaded</i> version that takes a value, not a position).

`names.size();`

Symbol	Meaning
<code>.size()</code>	Returns the current number of elements in the list, as an <code>int</code> .

Step 5: Keyword Explanation

Generics `<String>` (introductory explanation — full depth comes in Module 10)

- **What it means:** A way to tell a class "you will work with this specific type of data," checked by the compiler.
- **Why it exists:** Before generics (pre-Java 5), collections held plain `Object` references, meaning you could accidentally add a `String` and an `Integer` to the same list, and you'd get a runtime crash when retrieving and using them incorrectly. Generics catch this mistake **at compile time** instead.
- **When to use it:** Always specify a type when declaring a collection — `ArrayList<String>`, `ArrayList<Integer>`, etc. Avoid raw, untyped collections.

`ArrayList` (the class)

- **What it means:** A resizable array implementation of the `List` interface.
- **Why it exists:** To give you array-like indexed access (`get`, `set`) **plus** the ability to grow/shrink dynamically, plus built-in utility methods (search, remove, sort, etc.).
- **When to use it:** When you need an ordered collection, will access elements frequently by index, and don't need every operation to be lightning-fast for insertions/removals in the *middle* of the list (we'll compare this to `LinkedList` next).

`.add()`

- **What it means:** Method to insert an element.
- **Why it exists:** Arrays have no such method — you'd have to manually track an index and assign `array[i] = value`. `ArrayList` handles all of that internally.
- **When to use it:** Anytime you want to insert a new element, either at the end (`add(value)`) or at a specific position (`add(index, value)` — an overloaded version).

`.get()`

- **What it means:** Retrieves an element by index.
- **When to use it:** Whenever you need to read a specific position's value.

`.remove()`

- **What it means:** Deletes an element — either by value (`remove(Object)`) or by index (`remove(int index)`), these are **overloaded** methods (Module 4 concept).
- **Careful:** `remove(1)` removes the element *at index 1*. `remove(Integer.valueOf(1))` would remove the value `1` itself if it's an `ArrayList<Integer>`. This ambiguity is a classic beginner trap — covered below.

`.size()`

- **What it means:** Returns the number of elements currently stored.
- **Why it's different from arrays:** Arrays use a **field** called `.length` (no parentheses, not a method). `ArrayList` uses a **method** called `.size()` (with parentheses). This inconsistency trips up almost every beginner — memorize it now.

Step 6: Compiler Behavior

- The compiler enforces the generic type: `names.add(42);` on an `ArrayList<String>` will **fail to compile** — `42` is an `int` / `Integer`, not a `String`.
- The compiler resolves `.remove(...)` to the correct overload based on the argument's type — if you pass an `int` literal, it picks `remove(int index)`; if you pass an `Integer` object or `String`, it picks `remove(Object o)`.

Step 7: Runtime Behavior

This is genuinely important to understand, not just memorize: **how does `ArrayList` actually grow?**

1. Internally, `ArrayList` is backed by a real Java **array** (the same array type from Module 5) — but it's hidden from you.
2. When you create `new ArrayList<>()`, Java allocates a small internal array (commonly capacity 10, though this is an implementation detail).
3. When you `.add()` an element and the internal array is full, `ArrayList` automatically:
 - Creates a **new, bigger array** internally (typically 1.5× the size).
 - Copies all existing elements over.
 - Adds the new element.
4. This resizing happens **transparently** — you never see it, but it's worth knowing it happens "under the hood," because it has a performance cost.

Step 8: Memory Behavior

- The `ArrayList` object itself lives on the **heap**.
- It holds a **reference** to an internal array (also on the heap).
- When resizing occurs, the **old internal array becomes unreferenced** and is eligible for garbage collection, while a new larger array takes its place.
- Each element stored is a **reference** to an object (Strings, custom objects, etc.) — `ArrayList` cannot directly store primitives like `int` or `double`. This leads to our next important concept.

Autoboxing (a "first-time" concept worth flagging)

- **What it means:** Java automatically converts a primitive type (`int`) into its corresponding **wrapper class** (`Integer`) when needed — for example, when adding it to a collection.
- **Why it exists:** Collections only work with **objects**, not primitives. Since `int` is a primitive (Module 1), Java needs a way to "wrap" it as an object: `Integer`. Autoboxing makes this conversion automatic instead of requiring you to write `Integer.valueOf(5)` manually every time.
- **Example:**

```
java
```

```
ArrayList<Integer> numbers = new ArrayList<>();  
numbers.add(5); // Java auto-converts int 5 into Integer.valueOf(5)
```

- **Cost:** Autoboxing has a small performance overhead compared to raw primitive arrays, since each value becomes a full object on the heap instead of a raw

primitive value.

Step 9: Common Mistakes

1. Using `.length` instead of `.size()`

java

```
names.length(); // WRONG - compiler error, ArrayList has no .length
names.size();   // correct
```

2. The remove-by-index vs remove-by-value ambiguity

java

```
ArrayList<Integer> nums = new ArrayList<>();
nums.add(10);
nums.add(20);
nums.add(30);
nums.remove(1);           // removes the element AT index 1 → removes 20
nums.remove(Integer.valueOf(20)); // removes the VALUE 20 explicitly
```

Beginners assume `remove(20)` removes the value `20`, but since `20` is an `int` literal, Java picks the `remove(int index)` overload — removing whatever is at index `20`, which may throw an `IndexOutOfBoundsException` if the list isn't that long!

3. Trying to store primitives directly in the generic type

java

```
ArrayList<int> numbers = new ArrayList<>(); // COMPILER ERROR
ArrayList<Integer> numbers = new ArrayList<>(); // correct – must use wrapper
```

Generics **cannot** use primitive types — only object types/wrapper classes.

4. Modifying a list while iterating over it with a for-each loop

java

```
for (String name : names) {
    if (name.equals("Bob")) {
        names.remove(name); // throws ConcurrentModificationException at runtime
    }
}
```

```
}  
}
```

This is a very common real-world bug. We'll cover the correct way to do this (using an `Iterator`) shortly.

5. Forgetting to import `java.util.ArrayList`

Without the `import`, the compiler won't recognize `ArrayList` at all.

Type `next` to continue with `LinkedList`, where we'll compare it directly against `ArrayList` so you understand exactly when to choose one over the other.

next

MODULE 9 (continued): LinkedList

Step 3: Syntax

```
java  
  
import java.util.LinkedList;  
  
public class Main {  
    public static void main(String[] args) {  
        LinkedList<String> tasks = new LinkedList<>();  
  
        tasks.add("Write report");  
        tasks.addFirst("Check email");  
        tasks.addLast("Submit form");  
  
        System.out.println(tasks);  
        System.out.println(tasks.getFirst());  
        System.out.println(tasks.getLast());  
  
        tasks.removeFirst();  
        System.out.println(tasks);  
    }  
}
```

Output:

```
[Check email, Write report, Submit form]
Check email
Submit form
[Write report, Submit form]
```

Step 4: Breakdown

```
LinkedList<String> tasks = new LinkedList<>();
```

Structurally identical to `ArrayList`'s declaration — same generics, same diamond operator, same heap allocation. The difference is entirely **internal**, in how `LinkedList` stores its data — which we'll get to in a moment.

```
tasks.add("Write report");
```

Same `.add()` method as before — inherited from the shared `List` interface. Adds to the end by default.

```
tasks.addFirst("Check email");
```

Symbol	Meaning
<code>.addFirst(value)</code>	A method unique to <code>LinkedList</code> (not available on <code>ArrayList</code>) — inserts the element at the very beginning.

```
tasks.addLast("Submit form");
```

Symbol	Meaning
<code>.addLast(value)</code>	Inserts at the very end — functionally similar to <code>.add()</code> , but more explicit about intent.

```
tasks.getFirst() / tasks.getLast()
```

Retrieve the first/last elements directly, without needing to know the index or call `.size() - 1`.

```
tasks.removeFirst();
```

Removes the first element from the list.

Step 5: Keyword Explanation

LinkedList

- **What it means:** A list implementation where each element is stored in a separate **node**, and each node holds a reference to the *next* (and *previous*) node — like a chain.
- **Why it exists:** `ArrayList`'s resizing (copying the whole array when full) and shifting elements during insert/delete in the middle can be costly. `LinkedList` solves specific use cases — frequent insertions/removals at the **beginning** or **middle** — much more efficiently.
- **When to use it:** When your program frequently adds/removes elements from the front or middle of the list, or when you're using it as a **queue** or **deque** (double-ended queue) structure.

`.addFirst()` / `.addLast()` / `.getFirst()` / `.getLast()` / `.removeFirst()` / `.removeLast()`

- **What they mean:** Convenience methods specific to `LinkedList` (technically part of the `Deque` interface, which `LinkedList` also implements) for operating on either end of the list directly.
- **Why they exist:** These operations are **fast** ($O(1)$ — constant time) for a linked structure, because there's no shifting of other elements involved — you just update a couple of node references.

Step 6: The Internal Structure (Critical Conceptual Difference)

This is the heart of understanding `LinkedList` vs `ArrayList`. Let's visualize it.

ArrayList internally:

Index:	0	1	2
	[Alice]	[Bob]	[Charlie]

A single, contiguous block of memory. To insert at index 0, **everything else must shift right by one** — an expensive operation if the list is large.

LinkedList internally:

[Check email]	<->	[Write report]	<->	[Submit form]
(Node 1)		(Node 2)		(Node 3)

Each **node** is a separate object containing:

1. The actual data ("Check email").
2. A reference to the **next** node.
3. A reference to the **previous** node.

To insert a new node at the front, you only need to:

1. Create a new node.
2. Point its `next` reference to the old first node.
3. Update the list's internal "first node" reference to point to the new node.

No shifting of existing elements is required. This is why `addFirst()` is fast on a `LinkedList` but slow on an `ArrayList`.

Step 7: Compiler Behavior

- Both `ArrayList` and `LinkedList` implement the same `List` interface, so the compiler treats them as interchangeable wherever a `List<String>` type is expected:

java

```
List<String> list1 = new ArrayList<>(); // valid
List<String> list2 = new LinkedList<>(); // also valid
```

This is **polymorphism** applied to collections (Module 7 concept) — and it's actually considered **best practice** to declare variables using the interface type (`List`) rather than the concrete class, so you can swap implementations later without changing other code.

- However, `.addFirst()`, `.addLast()`, `.getFirst()`, `.getLast()` are **not** part of the `List` interface — they belong to `Deque`. If you declare your variable as `List<String> tasks = new LinkedList<>();`, the compiler will **reject** `tasks.addFirst(...)` because it only knows about `List`'s methods, even though the actual object is a `LinkedList`. You'd need to declare it as `LinkedList<String>` or `Deque<String>` to access those methods.
-

Step 8: Runtime Behavior

Time Complexity Comparison (essential to memorize)

Operation	ArrayList	LinkedList
Get by index (<code>get(i)</code>)	Fast — $O(1)$, direct memory calculation	Slow — $O(n)$, must walk node-by-node from the start
Add/remove at the end	Fast (usually) — $O(1)$	Fast — $O(1)$
Add/remove at the beginning	Slow — $O(n)$, must shift everything	Fast — $O(1)$
Add/remove in the middle	Slow — $O(n)$, shifting required	Slow — $O(n)$, must walk to that node first

This table is genuinely one of the most important things to memorize in this entire module. It directly answers "which one should I use?"

- **Frequent random access by index** → **ArrayList**.
- **Frequent insertion/removal at the ends** → **LinkedList**.

Step 9: Memory Behavior

- **ArrayList** uses one **contiguous block** of memory (the internal array) — generally more memory-efficient because there's no overhead per element beyond the array slots themselves.
- **LinkedList** allocates a **separate node object** for every single element. Each node stores not just the data, but also two references (`next` and `previous`). This means **LinkedList** uses **noticeably more memory per element** than **ArrayList**.
- Because **LinkedList** nodes are scattered across the heap (not contiguous), accessing them sequentially is also slower at the hardware level due to **cache locality** — **ArrayList**'s contiguous memory plays better with CPU cache behavior. (This is a deeper performance detail — good to know exists, not essential to master yet.)

Step 10: Common Mistakes

1. Declaring the variable as **List** but trying to use **LinkedList**-specific methods

java

```
List<String> tasks = new LinkedList<>();
tasks.addFirst("X"); // COMPILER ERROR - addFirst() not in List interface
```

Fix: declare as `LinkedList<String>` or `Deque<String>` if you need those specific methods.

2. Using `LinkedList` by default without considering access patterns

Many beginners assume `LinkedList` is "more advanced" so it must be "better." In practice, `ArrayList` is used far more often in real Java code because random access and iteration are far more common than frequent front/middle insertion.

3. Using `get(i)` in a loop on a `LinkedList`

java

```
LinkedList<String> list = new LinkedList<>();
// ... assume many elements added
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i)); // SLOW - O(n) each call, O(n²) total!
}
```

For a `LinkedList`, this pattern is a serious performance trap. Use a **for-each loop** or an **Iterator** instead — both traverse the list's nodes sequentially without restarting from the beginning each time.

4. Forgetting that `LinkedList` also implements `Queue` and `Deque`

This makes it directly usable as a queue/stack-like structure, which we'll explore properly in the upcoming Stack/Queue sections.

Type `next` to continue with `Stack`.

next

MODULE 9 (continued): Stack

Step 3: Syntax

java

```
import java.util.Stack;

public class Main {
    public static void main(String[] args) {
```

```

Stack<String> books = new Stack<>();

books.push("Java Basics");
books.push("Data Structures");
books.push("Algorithms");

System.out.println(books);
System.out.println(books.peek());

books.pop();
System.out.println(books);
System.out.println(books.isEmpty());
}
}

```

Output:

```

[Java Basics, Data Structures, Algorithms]
Algorithms
[Java Basics, Data Structures]
false

```

Step 4: Breakdown

```
Stack<String> books = new Stack<>();
```

Same structural pattern you've now seen three times — generic type, diamond operator, heap allocation. The behavioral difference is entirely in *how elements are added and removed*.

```
books.push("Java Basics");
```

Symbol

Meaning

```
.push(value)
```

Adds an element **to the top** of the stack.

```
books.peek();
```

Symbol

Meaning

```
.peek()
```

Looks at the **top** element **without removing it**.

`books.pop();`

Symbol

Meaning

`.pop()`

Removes **and returns** the top element.

`books.isEmpty();`

Symbol

Meaning

`.isEmpty()`

Returns a `boolean` — `true` if the stack has zero elements.

Step 5: Keyword Explanation

`Stack`

- **What it means:** A collection that follows the **LIFO** principle — **Last In, First Out**.
- **Why it exists:** Many real-world problems naturally behave this way — think of a physical stack of plates. You can only easily add or remove from the **top**. Examples in programming: the "undo" feature in an editor, the call stack itself (yes, the same call stack from Module 8's exception propagation!), navigating browser history (back button).
- **When to use it:** Whenever the *most recently added* item needs to be the *first* one processed/removed.

`.push()`

- **What it means:** Adds to the top.
- **Why "push" and not "add"?** Naming convention specific to stack-like structures — it emphasizes the LIFO mental model. `Stack` actually inherits `.add()` too (since `Stack` extends `Vector`, an older class), but `.push()` is the idiomatic, intention-revealing choice.

`.pop()`

- **What it means:** Removes the top item and gives it back to you.
- **When to use it:** Whenever you're "undoing" the most recent addition, or processing items in reverse order of insertion.

`.peek()`

- **What it means:** "Show me the top item, but don't touch the stack."

- **Why it exists:** Sometimes you want to **check** what's next without committing to removing it — important when you need to make a decision first.

Step 6: Visualizing LIFO Behavior

```
push("Java Basics")      → [Java Basics]
push("Data Structures")   → [Java Basics, Data Structures]
push("Algorithms")        → [Java Basics, Data Structures, Algorithms]
                           ↑ TOP

peek() → "Algorithms"    (just looks, doesn't remove)

pop()  → removes "Algorithms", returns it
       → [Java Basics, Data Structures]
                           ↑ TOP (now "Data Structures")
```

The key mental model: **whatever went in LAST comes out FIRST.**

Step 7: An Important Honest Note — `Stack` is Considered Legacy

Here's something most tutorials skip, but you deserve to know as you move toward advanced Java: the `java.util.Stack` class is **technically outdated**. It was part of Java's original 1.0 release (before the modern Collections Framework existed) and extends an old class called `Vector`, which is **synchronized** (thread-safe) by default — something that adds unnecessary performance overhead in single-threaded programs (we'll explain "synchronized" properly in Module 12).

Modern, recommended practice is to use a `Deque` (which `LinkedList` or `ArrayDeque` implements) for stack-like behavior instead:

```
java

import java.util.Deque;
import java.util.ArrayDeque;

Deque<String> books = new ArrayDeque<>();
books.push("Java Basics");
books.push("Data Structures");
System.out.println(books.peek());
books.pop();
```

This does the exact same job but with better performance, since `ArrayDeque` isn't burdened with unnecessary synchronization.

For learning purposes, your roadmap lists `Stack`, so we've covered it properly — but in real production code, prefer `Deque` / `ArrayDeque` for stack behavior. I'll flag this kind of "legacy vs. modern" distinction whenever it's relevant, since it's exactly the kind of practical knowledge that separates textbook Java from professional Java.

Step 8: Compiler Behavior

- `Stack<String>` enforces the generic type just like `ArrayList` / `LinkedList` — attempting `books.push(42)` won't compile.
 - Since `Stack` extends `Vector`, and `Vector` implements `List`, a `Stack` can also be assigned to a `List` reference — but then you'd lose access to `.push()` / `.pop()` / `.peek()`, the same interface-visibility issue you saw with `LinkedList`.
-

Step 9: Runtime Behavior

- `.push()` internally calls `.addElement()` (an old `Vector` method) which appends to the end of the internal array — and since `Stack` treats "the end of the array" as "the top," this is $O(1)$, fast.
 - `.pop()` removes from the end of the internal array — also $O(1)$.
 - Because `Vector`'s methods are `synchronized`, every single `.push()` and `.pop()` call involves acquiring and releasing a **lock** — even when you're not using multiple threads at all. This is pure overhead in a single-threaded context, which is the core performance criticism mentioned above.
-

Step 10: Memory Behavior

- Internally backed by a resizable array (like `ArrayList`), so similar resizing behavior applies — when full, a new larger array is allocated and old data copied over.
 - The "lock" used for synchronization is a property of the object itself (every Java object has an intrinsic lock available, a concept we'll fully explore in Module 12's synchronization section) — it doesn't cost significant extra memory, but it does cost CPU time on every call.
-

Step 11: Common Mistakes

1. Calling `.pop()` on an empty stack

java

```
Stack<String> stack = new Stack<>();  
stack.pop(); // throws EmptyStackException at runtime!
```

Always check `.isEmpty()` first, or wrap in a try-catch if appropriate.

2. Confusing `.peek()` with `.pop()`

Beginners often use `.peek()` when they actually meant to remove the element too, leading to logic bugs where the same item is processed repeatedly.

3. Assuming `Stack` is the "correct"/only way to get stack behavior in Java

As explained above, `ArrayDeque` is generally preferred in modern code.

4. Forgetting that `Stack` prints top-to-bottom in insertion order, not reverse

java

```
System.out.println(books); // [Java Basics, Data Structures, Algorithms]
```

Beginners sometimes expect the *top* element to print *first* in the list representation, but `toString()` just shows insertion order — the top is conceptually the **last** element shown, not the first.

Type `next` to continue with `Queue`.

next

MODULE 9 (continued): Queue

Step 3: Syntax

java

```
import java.util.Queue;  
import java.util.LinkedList;  
  
public class Main {  
    public static void main(String[] args) {  
        Queue<String> customers = new LinkedList<>();  
    }  
}
```

```

        customers.offer("Alice");
        customers.offer("Bob");
        customers.offer("Charlie");

        System.out.println(customers);
        System.out.println(customers.peek());

        customers.poll();
        System.out.println(customers);
    }
}

```

Output:

```

[Alice, Bob, Charlie]
Alice
[Bob, Charlie]

```

Step 4: Breakdown

```
Queue<String> customers = new LinkedList<>();
```

Look closely at this line — it's doing something new.

Symbol	Meaning
<code>Queue<String></code>	The declared type — <code>Queue</code> is an interface , not a class. You cannot do <code>new Queue<>()</code> .
<code>customers</code>	Variable name.
<code>new LinkedList<>()</code>	The actual object being created — <code>LinkedList</code> is a <i>class</i> that implements the <code>Queue</code> interface.

This is **polymorphism** (Module 7) in action again, just like we briefly saw with `List`. You're declaring the variable using the **general interface type** (`Queue`), but the actual object underneath is a concrete `LinkedList`. This means `customers` can only call methods that `Queue` defines — not `LinkedList`-specific ones like `.addFirst()` — unless you cast it.

```
customers.offer("Alice");
```

Symbol	Meaning
<code>.offer(value)</code>	Adds an element to the end (back) of the queue.

`customers.peek();`

Symbol	Meaning
<code>.peek()</code>	Looks at the element at the front of the queue, without removing it.

`customers.poll();`

Symbol	Meaning
<code>.poll()</code>	Removes and returns the element at the front of the queue.

Step 5: Keyword Explanation

Queue

- **What it means:** An **interface** representing a collection that follows the **FIFO** principle — First In, First Out.
- **Why it exists:** Real-world analogy: a checkout line at a store. The first person to join the line is the first person served. Programming use cases: task scheduling, print job queues, handling requests in the order they arrive, breadth-first search algorithms (you'll meet this in advanced data structures).
- **When to use it:** Whenever processing order **must match arrival order**.

`.offer()`

- **What it means:** Adds an element to the queue (specifically, to the back/end).
- **Why "offer" instead of "add"?** `Queue` actually has both `.add()` and `.offer()` — they do almost the same thing, but differ in **failure behavior**. `.add()` throws an exception if the queue has a fixed capacity and is full. `.offer()` returns `false` instead of throwing an exception. For most basic queues (like `LinkedList`-backed ones with no capacity limit), this distinction rarely matters in practice, but it matters greatly for capacity-bounded queues.

`.poll()`

- **What it means:** Removes and returns the front element. If the queue is **empty**, `.poll()` returns `null` instead of throwing an exception.

- **Compare to** `.remove()`: `Queue` also has a `.remove()` method that does the same removal — but **throws an exception** if the queue is empty, instead of returning `null`. This mirrors the `.add()` vs `.offer()` pattern: one throws on failure, the other fails "softly."

`.peek()`

- **What it means:** Same concept as `Stack`'s `.peek()` — look without removing — except here it looks at the **front**, not the top.

Step 6: Visualizing FIFO Behavior

```
offer("Alice")    → [Alice]
offer("Bob")      → [Alice, Bob]
offer("Charlie")  → [Alice, Bob, Charlie]
                   ↑ FRONT      ↑ BACK

peek() → "Alice"   (looks at front, doesn't remove)

poll() → removes "Alice", returns it
       → [Bob, Charlie]
         ↑ FRONT (now "Bob")
```

The key mental model: **whatever went in FIRST comes out FIRST**. This is the *exact opposite* ordering principle from `Stack`'s LIFO behavior — make sure these two stay distinct in your mind.

Step 7: Why `LinkedList` for a `Queue`?

You might wonder: why use `LinkedList` to implement `Queue` behavior, when we just spent the last section on `Stack`?

Recall from the `LinkedList` section: **adding/removing from the front is $O(1)$ — fast** for a `LinkedList`, because no shifting is required (unlike `ArrayList`, where removing from the front requires shifting every other element left by one).

Since a `Queue` constantly removes from the front (`poll()`) and adds to the back (`offer()`), `LinkedList`'s node-based structure is a **natural fit**. This is a perfect real-world example of *why* understanding internal data structure behavior (from the earlier `ArrayList` vs `LinkedList` comparison) actually matters when choosing implementations.

Step 8: Compiler Behavior

- `Queue` being declared as an interface means the compiler restricts `customers` to only the methods defined in the `Queue` interface (plus anything inherited from `Collection`). Even though the real object is a `LinkedList`, you cannot call `customers.addFirst(...)` without first casting:

java

```
((LinkedList<String>) customers).addFirst("Zara"); // legal, but rarely good p
```

- The compiler checks generic type consistency as always — `customers.offer(42)` won't compile for a `Queue<String>`.

Step 9: Runtime Behavior

- `.offer()` on a `LinkedList`-backed queue calls the internal `addLast()` logic — $O(1)$, since it just updates the tail node's reference.
- `.poll()` calls the internal `removeFirst()` logic — also $O(1)$, updating the head node's reference, no shifting involved.
- If the queue is empty and you call `.poll()`, it returns `null` rather than throwing — this is a **deliberate design choice** distinguishing `Queue`'s "soft failure" methods from `Stack`'s `.pop()`, which throws `EmptyStackException`.

Step 10: Memory Behavior

Identical underlying memory behavior to `LinkedList`, since that's the actual implementation being used here — each element is a separate heap-allocated node with `next`/`previous` references. No new memory concepts here; this is purely a different **interface view** on the same underlying structure.

Step 11: Common Mistakes

1. Trying to instantiate `Queue` directly

java

```
Queue<String> q = new Queue<>(); // COMPILER ERROR — Queue is an interface
```

You must instantiate a concrete implementing class (`LinkedList`, `ArrayDeque`, `PriorityQueue`, etc.).

2. Confusing `Queue`'s FIFO order with `Stack`'s LIFO order

This is the single most common conceptual mix-up in this entire module. If you're ever unsure, say it out loud: "Queue = line at a store, first come first served."

3. Using `.remove()` on an empty queue and being surprised by an exception

java

```
Queue<String> q = new LinkedList<>();
q.remove(); // throws NoSuchElementException – empty queue!
q.poll();   // returns null instead – safer for empty-checking logic
```

4. Forgetting `peek()` can return `null` on an empty queue, and then calling a method on that `null` result

java

```
String front = customers.peek(); // null, if queue is empty
front.length(); // NullPointerException!
```

Always check for `null` before using the result, if there's any chance the queue could be empty.

Type `next` to continue with `PriorityQueue`.

next

MODULE 9 (continued): PriorityQueue

Step 3: Syntax

java

```
import java.util.PriorityQueue;

public class Main {
    public static void main(String[] args) {
        PriorityQueue<Integer> numbers = new PriorityQueue<>();
    }
}
```

```

        numbers.offer(50);
        numbers.offer(10);
        numbers.offer(30);
        numbers.offer(5);

        System.out.println(numbers);
        System.out.println(numbers.peek());

        while (!numbers.isEmpty()) {
            System.out.println(numbers.poll());
        }
    }
}

```

Output:

```

[5, 10, 30, 50]
5
5
10
30
50

```

(Note: the printed order of the internal array via `toString()` isn't strictly sorted in general — it just happens to look sorted here because of how this particular small example is structured. The `.poll()` order, however, is **always** guaranteed sorted. More on this distinction below — it's important.)

Step 4: Breakdown

```
PriorityQueue<Integer> numbers = new PriorityQueue<>();
```

Same generic/diamond-operator pattern as before. The new concept is entirely **behavioral**: this is *not* a FIFO structure like a regular `Queue`. Instead, elements come out in order of **priority** — by default, the **smallest value first**.

```
numbers.offer(50); ... etc.
```

Same `.offer()` method you already know from `Queue` (since `PriorityQueue` implements `Queue`). But internally, it does something different from a plain `LinkedList`-backed queue — it doesn't just append to the end.

```
numbers.peek();
```

Returns the **highest-priority** element (by default, the smallest number) — **not** necessarily the one that was inserted first.

```
numbers.poll();
```

Removes and returns the highest-priority element. Called repeatedly here in a loop until the queue is empty.

Step 5: Keyword Explanation

PriorityQueue

- **What it means:** A queue where elements are **not** ordered by insertion order, but instead by a defined **priority** — by default, natural ordering (smallest first for numbers, alphabetical first for strings).
 - **Why it exists:** Many real-world problems need "process the most urgent/smallest/largest item next," not "process whatever arrived first." Examples: hospital emergency room triage (most critical patient first, not first-arrived), task schedulers (highest priority task runs first), Dijkstra's shortest-path algorithm (a famous algorithm you'll encounter in advanced data structures courses).
 - **When to use it:** Whenever you repeatedly need "give me the current minimum (or maximum) from a changing collection," efficiently.
-

Step 6: How Does It Know the "Priority"? — Natural Ordering

For `PriorityQueue<Integer>`, Java knows `5 < 10 < 30 < 50` because `Integer` implements an interface called `Comparable` — meaning Java already knows how to compare two `Integer` values to determine which is "smaller."

This is your **first real exposure** to a concept that becomes very important going forward, so let's name it properly.

Comparable (first-time interface)

- **What it means:** An interface with one method, `compareTo(other)`, that defines "how to compare this object to another object of the same type" — returning a negative number, zero, or positive number depending on whether `this` is less than, equal to, or greater than `other`.
- **Why it exists:** Java's built-in types like `Integer`, `String`, `Double` all implement `Comparable` so that sorting/ordering structures (`PriorityQueue`, `TreeSet`, `TreeMap`, `Collections.sort()`) know how to order them automatically.
- **When to use it:** When you create your **own custom class** (e.g., a `Student` class) and want objects of that class to have a natural sort order (e.g., sort students by

grade), you'll implement `Comparable` yourself. We'll do this hands-on shortly.

Custom Priority with `Comparator` (brief introduction)

What if you want a `PriorityQueue` of `Integer`s, but **largest first** instead of the default smallest-first? You provide a `Comparator` — an object that defines a *custom* comparison rule, instead of relying on the default `Comparable` behavior.

```
java

import java.util.PriorityQueue;
import java.util.Comparator;

PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

maxHeap.offer(50);
maxHeap.offer(10);
maxHeap.offer(30);

System.out.println(maxHeap.poll()); // 50 - largest comes out first now
```

Symbol

Meaning

`Comparator.reverseOrder()`

A **static method** (you learned `static` in Module 7) that returns a ready-made `Comparator` object which reverses natural ordering.

`new PriorityQueue<>`
`(Comparator.reverseOrder())`

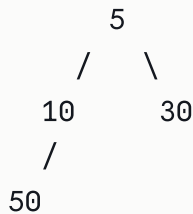
The **constructor** here is **overloaded** (Module 4 concept) — this version accepts a `Comparator` to customize ordering, instead of using default natural ordering.

We'll go much deeper into writing your own custom `Comparator`s when we reach sorting collections with custom objects — for now, just recognize this pattern exists.

Step 7: The Internal Structure — Binary Heap (Conceptual Overview)

This is genuinely useful to understand at a conceptual level, even though you won't need to implement it yourself right now.

`PriorityQueue` is internally backed by a data structure called a **binary heap** — specifically, a **min-heap** by default. Picture it as a tree-shaped arrangement (though stored internally as an array) where:



The rule: every parent node is **smaller than or equal to** both its children. This guarantees the **smallest element is always at the root** (the top), making `.peek()` instant — it just looks at the root.

When you `.poll()`:

1. The root (smallest) is removed and returned.
2. The heap **rearranges itself** to maintain the min-heap property — this involves moving the last element to the root position, then "sifting it down" by repeatedly swapping it with its smaller child until the heap rule is restored.

This rearrangement is why `.poll()` is $O(\log n)$ — not instant, but still much faster than scanning the entire collection for the minimum every time (which would be $O(n)$).

This also explains the earlier note: **the internal array order** (what `toString()` shows) **is NOT a fully sorted list** — it only guarantees the heap property (parent $\leq$ children), not a complete left-to-right sort. Only repeatedly calling `.poll()` gives you a guaranteed fully sorted sequence.

Step 8: Compiler Behavior

- The compiler requires that whatever type you use as the generic parameter either:
 - Implements `Comparable` (so natural ordering is available), **or**
 - You provide a `Comparator` explicitly in the constructor.
- If you try `new PriorityQueue<SomeCustomClass>()` where `SomeCustomClass` implements **neither**, this compiles fine (generics don't catch this at compile time), but it will throw a `ClassCastException` **at runtime** the moment you try to `.offer()` a second element and the heap tries to compare them.

Step 9: Runtime Behavior

Operation	Time Complexity	Why
<code>.offer()</code> (insert)	$O(\log n)$	New element is placed, then "sifted up" to maintain heap property
<code>.poll()</code> (remove min)	$O(\log n)$	Root removed, last element moved to root, then "sifted down"
<code>.peek()</code>	$O(1)$	Minimum is always at the root — no searching needed

Step 10: Memory Behavior

- Internally backed by an **array** (resizable, similar growth strategy to `ArrayList`), not actual tree node objects with pointers — the tree shape is purely conceptual; it's mathematically mapped onto array indices (parent of index `i` is at $(i-1)/2$, children at $2i+1$ and $2i+2$). This is more memory-efficient than a true node-based tree, since there's no per-node reference overhead.

Step 11: Common Mistakes

- Assuming `toString()` / iteration order is fully sorted

java

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(50); pq.offer(10); pq.offer(30);
System.out.println(pq); // NOT guaranteed to print [10, 30, 50] in that exact order
```

Only `.poll()` calls, used repeatedly, guarantee sorted retrieval order.

- Using a custom class without `Comparable` or a `Comparator`

java

```
class Student { String name; }
PriorityQueue<Student> pq = new PriorityQueue<>();
pq.offer(new Student());
pq.offer(new Student()); // ClassCastException at runtime!
```

- Forgetting `PriorityQueue` defaults to smallest-first (min-heap), not largest-first
Many beginners assume "priority" intuitively means "biggest/most important"

comes first," and are confused when `.poll()` returns the smallest number.

4. Trying to use `PriorityQueue` for simple FIFO behavior

If you don't actually need priority-based ordering, just use a plain

`Queue` / `LinkedList` — `PriorityQueue` has unnecessary $O(\log n)$ overhead for what could be an $O(1)$ operation.

Type `next` to continue with `HashSet`.

next

MODULE 9 (continued): HashSet

We're now moving from **List/Queue-family structures** (ordered, allow duplicates) to **Set-family structures** (no duplicates allowed). This is a meaningful shift in mental model, so let's build it carefully.

Step 3: Syntax

```
java

import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        HashSet<String> fruits = new HashSet<>();

        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Apple"); // duplicate attempt

        System.out.println(fruits);
        System.out.println(fruits.size());
        System.out.println(fruits.contains("Banana"));

        fruits.remove("Apple");
        System.out.println(fruits);
    }
}
```

Possible Output:

```
[Banana, Apple]
2
true
[Banana]
```

Step 4: Breakdown

```
HashSet<String> fruits = new HashSet<>();
```

Same generic/diamond pattern. The behavioral difference begins at `.add()`.

```
fruits.add("Apple");
```

 (called twice with the same value)

Symbol	Meaning
<code>.add(value)</code>	Same method name as <code>ArrayList</code> , but here it silently does nothing if the value already exists in the set — no exception, no error, it just doesn't add a second copy.

```
fruits.size();
```

Returns `2`, not `3` — even though `.add("Apple")` was called twice. The duplicate was rejected.

```
fruits.contains("Banana");
```

Symbol	Meaning
<code>.contains(value)</code>	Returns <code>true</code> / <code>false</code> depending on whether the value exists in the set.

Step 5: Keyword Explanation

HashSet

- **What it means:** A collection that stores **unique** elements only, with **no guaranteed order**.
- **Why it exists:** Sometimes you don't care about order at all — you only care "does this value exist, and can I quickly check?" or "give me only the distinct values." Lists allow duplicates and care about order; sets solve the opposite problem.
- **When to use it:** Removing duplicates from data, fast existence-checking (e.g., "has this username already been used?"), set-theory operations (union, intersection).

Why "no guaranteed order"? — Hashing (the core mechanism)

This is the most important concept in this section. Let's build it from the ground up.

When you call `fruits.add("Apple")`:

1. Java calls a method called `.hashCode()` on `"Apple"`. Every Java object has this method (inherited from the universal `Object` class — recall from Module 7 that *all* classes ultimately extend `Object`).
2. `.hashCode()` returns an `int` — a number computed from the object's contents, intended to act like a "fingerprint."
3. `HashSet` uses this number to decide **which internal "bucket"** to store the element in — imagine the `HashSet` as a row of many small buckets, and the hash code (after some math) determines bucket number.
4. When you later call `.contains("Apple")`, Java recomputes `"Apple"`'s hash code, jumps **directly** to that bucket, and checks if `"Apple"` is actually there.

This is why `.contains()` and `.add()` are extremely fast — **O(1) on average** — Java doesn't need to scan every single element like a `List` would; it jumps straight to the right "neighborhood."

But this also explains *why there's no guaranteed order*: elements are placed based on their hash code's bucket location, **not** insertion order. The printed order can look almost arbitrary.

`.hashCode()` and `.equals()` — A Critical Pair (first-time deep dive)

- **What they mean:**
 - `.hashCode()` — produces an integer fingerprint of an object, used to find the right bucket.
 - `.equals()` — determines if two objects are *actually* the same value (you may recall using `.equals()` for `String` comparison back in Module 6).
- **Why they exist together:** `HashSet` first uses `.hashCode()` to jump to a bucket *quickly*, but multiple different objects can sometimes produce the **same** hash code (called a **collision**). So once it's in the right bucket, it uses `.equals()` to confirm "is this *really* the same value, or just a coincidental hash collision?"
- **The Golden Rule:** If you ever override `.equals()` on your own custom class, you **must** also override `.hashCode()` consistently — meaning, if two objects are `.equals()`, they **must** produce the same `.hashCode()`. Breaking this rule causes `HashSet` (and `HashMap`, coming soon) to behave incorrectly — duplicates might sneak in, or `.contains()` might wrongly return `false` for something that's actually there.

We'll get hands-on with overriding these methods properly once we reach `HashMap`, since the demonstration is clearer there — but the concept starts here.

Step 6: Compiler Behavior

- Standard generic type enforcement, as with every collection so far.
 - The compiler does **not** check whether your custom class properly overrides `.hashCode()` / `.equals()` — this is a **runtime correctness concern**, not a compile-time one. Forgetting to override them on a custom class won't stop your code from compiling, but it can produce confusing bugs (duplicates not being detected, etc.) at runtime, because Java falls back to the default `Object` implementations, which compare by **memory reference**, not by content.
-

Step 7: Runtime Behavior

Walking through what happens internally for `fruits.add("Apple")` called twice:

1. First call: `"Apple".hashCode()` is computed → say it maps to bucket #7. Bucket #7 is empty, so `"Apple"` is inserted there.
2. Second call (duplicate attempt): `"Apple".hashCode()` is computed again → same value, same bucket #7.
3. `HashSet` checks bucket #7, finds `"Apple"` already there, calls `.equals()` to confirm — it matches.
4. Since it's confirmed to be a duplicate, **the add is silently ignored**. `.add()` actually returns a `boolean` (`true` if added, `false` if it was a duplicate) — though in our example, we didn't capture/check that return value.

```
java
```

```
boolean wasAdded = fruits.add("Apple");  
System.out.println(wasAdded); // false, second time — already existed
```

Step 8: Memory Behavior

- Internally, `HashSet` is built on top of a `HashMap` (we'll see `HashMap` next — this will make more sense then). Each element you add is stored as a "key" in that underlying map, with a dummy placeholder value.
- The bucket structure means memory isn't a single contiguous block (unlike `ArrayList`) — it's an array of buckets, where each bucket can itself hold a small list of colliding elements.
- As more elements are added, `HashSet` automatically **resizes** its internal bucket array (similar resizing concept to `ArrayList`, just applied to the bucket array) to

keep the average number of elements-per-bucket low, preserving that fast $O(1)$ average performance.

Step 9: Common Mistakes

1. Expecting insertion order to be preserved

java

```
HashSet<String> set = new HashSet<>();
set.add("Zebra");
set.add("Apple");
set.add("Mango");
System.out.println(set); // order is unpredictable, NOT [Zebra, Apple, Mango]
```

If you need insertion order preserved *and* uniqueness, use `LinkedHashSet` instead (a close cousin we haven't covered explicitly, but worth knowing exists).

2. Using `HashSet` with custom objects without overriding `equals()` / `hashCode()`

java

```
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}

HashSet<Point> points = new HashSet<>();
points.add(new Point(1, 2));
points.add(new Point(1, 2)); // adds a SECOND "duplicate" — because no equals
System.out.println(points.size()); // 2, not 1 — likely NOT what you wanted
```

Without overriding, Java uses `Object`'s default `.equals()`, which only returns `true` if it's *literally the same object in memory* — two separately created `Point` objects with identical field values are still considered "different."

3. Assuming `.add()` throws an exception on duplicates

It doesn't — it just silently returns `false` and ignores the duplicate. Beginners sometimes wrap `.add()` in a try-catch expecting an exception that will never come.

4. Iterating a `HashSet` expecting consistent order across multiple runs

The order can even change between different runs of the *same* program, depending on JVM internals — never rely on it.

Type `next` to continue with `TreeSet`.

next

MODULE 9 (continued): TreeSet

Step 3: Syntax

```
java

import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        TreeSet<Integer> numbers = new TreeSet<>();

        numbers.add(50);
        numbers.add(10);
        numbers.add(30);
        numbers.add(10); // duplicate attempt

        System.out.println(numbers);
        System.out.println(numbers.first());
        System.out.println(numbers.last());
        System.out.println(numbers.higher(10));
        System.out.println(numbers.lower(30));
    }
}
```

Output:

```
[10, 30, 50]
10
50
30
10
```

Step 4: Breakdown

```
TreeSet<Integer> numbers = new TreeSet<>();
```

Same generic/diamond pattern as every collection so far.

```
numbers.add(50); ... etc.
```

Same `.add()` method as `HashSet` — rejects duplicates silently. But notice the printed output: `[10, 30, 50]` — **sorted ascending order**, even though we inserted `50, 10, 30, 10` in that scrambled order. This is the entire point of `TreeSet`.

```
numbers.first() / numbers.last()
```

Symbol	Meaning
<code>.first()</code>	Returns the smallest element currently in the set.
<code>.last()</code>	Returns the largest element currently in the set.

```
numbers.higher(10) / numbers.lower(30)
```

Symbol	Meaning
<code>.higher(value)</code>	Returns the smallest element strictly greater than the given value.
<code>.lower(value)</code>	Returns the largest element strictly less than the given value.

Step 5: Keyword Explanation

`TreeSet`

- **What it means:** A `Set` implementation that **automatically keeps its elements sorted**, with no duplicates, at all times.
- **Why it exists:** `HashSet` gives you fast uniqueness-checking but zero ordering guarantees. Sometimes you need *both* — uniqueness *and* sorted order, automatically maintained even as you keep adding/removing elements. `TreeSet` solves exactly that.
- **When to use it:** Leaderboards, range queries ("give me everything between X and Y"), anything where sorted, de-duplicated data matters.

```
.first() / .last()
```

- **What they mean:** Direct access to the minimum/maximum element — both $O(\log n)$, since they're found by walking down one side of the underlying tree structure

(explained next), not by scanning everything.

`.higher()` / `.lower()` (and their cousins `.ceiling()` / `.floor()`)

- **What they mean:** "Navigation" methods unique to sorted structures — letting you ask relative questions about your data ("what comes right after this value?").
- **Quick distinction worth knowing:**

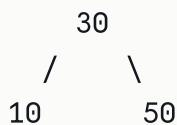
Method	Meaning
<code>.higher(x)</code>	Strictly greater than <code>x</code>
<code>.lower(x)</code>	Strictly less than <code>x</code>
<code>.ceiling(x)</code>	Greater than or equal to <code>x</code>
<code>.floor(x)</code>	Less than or equal to <code>x</code>

- **When to use them:** Useful for things like "find the next available time slot after 2:00 PM" or "find the closest price below my budget."

Step 6: The Internal Structure — Red-Black Tree (Conceptual Overview)

`TreeSet` is internally backed by a **Red-Black Tree** — a type of **self-balancing binary search tree**. You don't need to implement one yourself right now, but understanding the *shape* of it explains everything about `TreeSet`'s behavior.

Binary Search Tree — the core idea:



The rule: for any node, everything in its **left** subtree is smaller, everything in its **right** subtree is larger.

When you call `.add(10)`, Java:

1. Starts at the root (`30`).
2. Compares `10` vs `30` → smaller → go left.
3. Reaches an empty spot → inserts `10` there.

When you call `.first()`, Java just keeps going **left** from the root until there's nowhere left to go — that's guaranteed to be the smallest value. `.last()` does the mirror opposite, always going right.

Why "self-balancing" (Red-Black Tree specifically)?

A naive binary search tree can become lopsided if you insert already-sorted data (e.g., `10, 20, 30, 40...` in order) — it degenerates into something resembling a `LinkedList`, losing all performance benefits. A **Red-Black Tree** automatically performs internal rotations during insertion/removal to keep the tree roughly balanced — ensuring operations stay reliably fast ($O(\log n)$) no matter what order you insert data in.

This is genuinely advanced data-structures territory — for now, just remember:

`TreeSet` = sorted + no duplicates + reliably fast ($O(\log n)$), thanks to a self-balancing tree underneath.

Step 7: How Does `TreeSet` Know the Order? — Back to `Comparable`

This connects directly back to the `Comparable` concept from `PriorityQueue`.

`TreeSet<Integer>` works out-of-the-box because `Integer` implements `Comparable`. If you tried `TreeSet<SomeCustomClass>` without `Comparable` (or a `Comparator`), you'd get a runtime `ClassCastException` the moment a second element is added and a comparison is attempted.

```
java

import java.util.TreeSet;
import java.util.Comparator;

TreeSet<String> names = new TreeSet<>(Comparator.reverseOrder());
names.add("Charlie");
names.add("Alice");
names.add("Bob");
System.out.println(names); // [Charlie, Bob, Alice] - reverse alphabetical
```

Same overloaded-constructor pattern you saw with `PriorityQueue` — passing a `Comparator` overrides the default natural ordering.

Step 8: Compiler Behavior

- Standard generic enforcement applies.
 - As with `PriorityQueue`, the compiler does **not** verify at compile time that your generic type is actually `Comparable` — that failure only appears at **runtime**, the moment a comparison is actually attempted between two elements.
-

Step 9: Runtime Behavior — Performance Comparison Across All Three Sets/Lists So Far

This is a good moment to consolidate, since you now know enough structures to meaningfully compare them.

Operation	ArrayList	HashSet	TreeSet
Add	O(1) typical	O(1) average	O(log n)
Contains/Search	O(n)	O(1) average	O(log n)
Maintains order?	Insertion order	No order	Sorted order
Allows duplicates?	Yes	No	No

Decision guide:

- Need fast lookup, don't care about order or duplicates → `HashSet`.
- Need sorted, de-duplicated data → `TreeSet`.
- Need ordered, duplicate-allowing, index-accessible data → `ArrayList`.

Step 10: Memory Behavior

- Each element in a `TreeSet` is wrapped in a **tree node object**, holding references to its left child, right child, and (in Red-Black Trees specifically) a color marker (red or black) used internally for the balancing algorithm.
- This means `TreeSet` has **more memory overhead per element** than `HashSet`, similar to how `LinkedList` had more overhead than `ArrayList` — the price you pay for maintained order.

Step 11: Common Mistakes

1. Using `TreeSet` when you don't actually need sorted order

If you just need fast uniqueness checks and don't care about order, `HashSet` is simpler and slightly faster — don't pay for sorting you won't use.

2. Forgetting custom objects need `Comparable` or a `Comparator`

```
java
```

```
class Employee { String name; }  
TreeSet<Employee> employees = new TreeSet<>();
```

```
employees.add(new Employee()); // fine, only one element
employees.add(new Employee()); // ClassCastException! Can't compare two Emplo
```

3. Calling `.first()` or `.last()` on an empty `TreeSet`

java

```
TreeSet<Integer> empty = new TreeSet<>();
empty.first(); // throws NoSuchElementException at runtime!
```

Always check `.isEmpty()` first if there's any chance the set could be empty.

4. Assuming `.higher()` / `.lower()` are inclusive

They're **strict** (exclusive) — if you want inclusive behavior, you need `.ceiling()` / `.floor()` instead, as shown in the table above.

We've now covered the entire **List, Queue, and Set** branches of the Collections Framework. Next comes the **Map** branch — a completely separate hierarchy that stores **key-value pairs**, which is arguably the single most-used part of the entire Collections Framework in real-world Java code.

Type `next` to continue with `HashMap`.

next

MODULE 9 (continued): HashMap

This is one of the most important structures you'll learn in all of Java. Take your time with this one.

Step 3: Syntax

java

```
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
        HashMap<String, Integer> ages = new HashMap<>();
    }
}
```

```

    ages.put("Alice", 25);
    ages.put("Bob", 30);
    ages.put("Charlie", 35);
    ages.put("Alice", 26); // overwrite attempt

    System.out.println(ages);
    System.out.println(ages.get("Bob"));
    System.out.println(ages.containsKey("Charlie"));
    System.out.println(ages.containsValue(100));

    ages.remove("Bob");
    System.out.println(ages);

    for (String key : ages.keySet()) {
        System.out.println(key + " -> " + ages.get(key));
    }
}

```

Possible Output:

```

{Alice=26, Bob=30, Charlie=35}
30
true
false
{Alice=26, Charlie=35}
Alice -> 26
Charlie -> 35

```

Step 4: Breakdown

```
HashMap<String, Integer> ages = new HashMap<>();
```

This is your **first** collection with **TWO** generic type parameters. Let's slow down here.

Symbol	Meaning
HashMap	The class — stores data as key-value pairs .
<String, Integer>	Two generic types this time: the first (String) is the type of the keys , the second (Integer) is the type of the values .
ages	Variable name.

Symbol	Meaning
<code>new HashMap<></code> <code>()</code>	Creates a new, empty map on the heap.

Think of a `HashMap` like a real-world dictionary: each **key** ("Alice") maps to a **value** (25) — and just like a dictionary, you can't have the same word (key) defined twice; the second definition simply replaces the first.

```
ages.put("Alice", 25);
```

Symbol	Meaning
<code>.put(key, value)</code>	Inserts a new key-value pair, or if the key already exists, overwrites its value.

```
ages.put("Alice", 26);
```

 (called again with the same key)

This doesn't create a second entry — it **replaces** the value associated with "Alice" from 25 to 26. This is the critical difference from a `List`'s `.add()` — `.put()` with an existing key is an **update**, not an insertion.

```
ages.get("Bob");
```

Symbol	Meaning
<code>.get(key)</code>	Retrieves the value associated with the given key. Returns <code>null</code> if the key doesn't exist.

```
ages.containsKey("Charlie") / ages.containsValue(100)
```

Symbol	Meaning
<code>.containsKey(key)</code>	Checks if a given key exists.
<code>.containsValue(value)</code>	Checks if a given value exists anywhere in the map (slower — must check every entry, unlike <code>.containsKey()</code>).

```
ages.remove("Bob");
```

Removes the entry whose key is "Bob" — both the key and its associated value disappear together.

```
for (String key : ages.keySet())
```

Symbol	Meaning
<code>.keySet()</code>	Returns a <code>Set</code> (specifically, a view) containing all the keys in the map.
<code>for (String key : ...)</code>	A for-each loop (Module 3 concept) iterating over those keys.

```
key + " -> " + ages.get(key)
```

For each key retrieved from `.keySet()`, we use `.get(key)` again to fetch its corresponding value, then concatenate them into a readable string.

Step 5: Keyword Explanation

HashMap

- **What it means:** A collection storing data as **key-value pairs**, where each key is **unique**, using hashing (the same bucket-based mechanism from `HashSet`) for fast lookup.
- **Why it exists:** An enormous number of real-world problems are naturally "lookup by identifier" problems: username → user profile, product ID → product details, word → definition, ZIP code → city name. Lists/Sets don't model this relationship; `Map` does.
- **When to use it:** Whenever you need to associate one piece of data with another and retrieve it efficiently by that association — this is *extremely* common in real applications.

`.put()`

- **What it means:** Insert or update a key-value pair.
- **Why "put" instead of "add"?** Naming convention distinguishing map behavior (insert-or-replace) from list/set behavior (just insert).

`.get()`

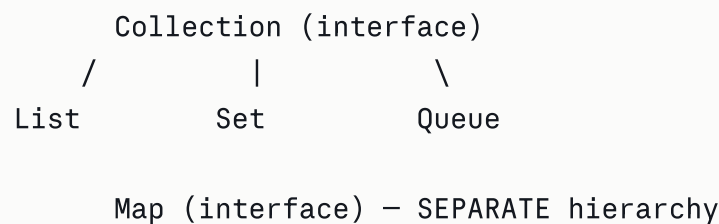
- **What it means:** Retrieve the value for a given key.
- **Important nuance:** If the key doesn't exist, `.get()` returns `null` — it does **not** throw an exception. This is a frequent source of `NullPointerException` bugs later if you don't check first.

`.keySet()`

- **What it means:** Gives you access to all keys as a `Set` (remember — keys are unique, so a `Set` is the natural fit).
- **Why it exists:** So you can iterate over a map's contents, since a `Map` itself is **not** directly iterable with a simple for-each loop (you can't write `for (x : ages)` directly — `Map` doesn't implement `Collection` or `Iterable`).

Step 6: Why `Map` Is NOT Part of the `Collection` Interface

This surprises a lot of beginners, so let's address it head-on, referring back to the hierarchy diagram from the start of this module:



`List`, `Set`, and `Queue` all represent a **single group of elements**. `Map` represents **pairs** of related elements — conceptually different enough that Java's designers gave it its own separate interface hierarchy, rather than forcing it to fit the `Collection` mold.

This is *why* you can't directly for-each loop over a `Map` — you must go through `.keySet()`, `.values()`, or `.entrySet()` (covered next) to get something iterable.

Step 7: Better Iteration — `.entrySet()`

The approach in our example (looping over keys, then calling `.get(key)` again inside the loop) **works**, but it's slightly inefficient — you're doing a second hash lookup for every key you already have. The more idiomatic, efficient approach:

```
java
import java.util.Map;

for (Map.Entry<String, Integer> entry : ages.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

Breakdown

Symbol	Meaning
<code>.entrySet()</code>	Returns a <code>Set</code> of <code>Map.Entry</code> objects — each one bundles a key AND its value together as a single object, avoiding a second lookup.
<code>Map.Entry<String, Integer></code>	The type of each bundled pair — <code>Map</code> is the outer interface, <code>Entry</code> is a nested interface inside it (we'll cover nested classes/interfaces formally in Module 14, but for now: think of it as "a type that only makes sense in the context of <code>Map</code> ").
<code>entry</code>	Loop variable — one key-value pair per iteration.
<code>entry.getKey()</code>	Retrieves just the key from this pair.
<code>entry.getValue()</code>	Retrieves just the value from this pair.

This avoids the redundant `.get(key)` call entirely, since you already have the value sitting right there in `entry`.

Step 8: Compiler Behavior

- The compiler enforces **both** generic types — `ages.put("David", "thirty")` won't compile, because `"thirty"` is a `String`, not an `Integer`, for a `HashMap<String, Integer>`.
- `Map.Entry<String, Integer>` must match the exact key/value types declared in the original `HashMap<String, Integer>` — mismatches are caught at compile time.

Step 9: Runtime Behavior

Internally, `.put("Alice", 25)` does the following:

- Computes `"Alice".hashCode()`.
- Maps that hash to a bucket index (using some internal math, typically `hashCode % numberOfBuckets`, though Java's actual implementation is a bit more refined).
- Checks if that bucket already has an entry with a key that `.equals()` `"Alice"`.
 - If yes → **overwrite** that entry's value.
 - If no → create a **new entry** (a key-value pair object) and place it in that bucket.

This is precisely why **the same** `.hashCode()` / `.equals()` rule from `HashSet` **applies here too** — in fact, more critically, since using a custom class as a **key** with broken

`.hashCode()` / `.equals()` can make `.get()` fail to find data you know you put in.

Step 10: Memory Behavior

- Each key-value pair is stored as an internal object (often literally called a "Node" or "Entry" in the JDK source code) holding: the key, the value, a reference to the next node (for handling collisions within the same bucket), and the cached hash code.
 - `HashSet`, as mentioned earlier, is literally implemented internally **using** a `HashMap` — each element you add to a `HashSet` becomes a key in a hidden internal `HashMap`, paired with a dummy constant value. This is why their performance characteristics are identical.
 - Like `HashSet`, the bucket array resizes automatically as more entries are added, to keep average lookups fast.
-

Step 11: Common Mistakes

- Using `.get()` and not handling a possible `null` return

java

```
Integer age = ages.get("David"); // David doesn't exist
int doubled = age * 2; // NullPointerException! age is null
```

Safer alternative:

java

```
Integer age = ages.getDefault("David", 0); // returns 0 if key missing
```

- Using a mutable custom object as a key, then changing it after insertion

java

```
HashMap<List<Integer>, String> map = new HashMap<>();
List<Integer> key = new ArrayList<>();
key.add(1);
map.put(key, "value");
key.add(2); // changing the key's contents AFTER it's been used as a key!
System.out.println(map.get(key)); // often returns null - the hash code chang
```

This is a subtle, dangerous bug: changing a key's contents after insertion changes its `.hashCode()`, so the map can no longer find it in its original bucket. **Rule: never use**

mutable objects as map keys unless you're certain they won't change.

3. Forgetting `Map` isn't directly iterable

java

```
for (String s : ages) { } // COMPILE ERROR - Map doesn't implement Iterable
```

Must use `.keySet()`, `.values()`, or `.entrySet()`.

4. Assuming insertion order is preserved

Same caveat as `HashSet` — no guaranteed order. If you need insertion order, use `LinkedHashMap`.

Type `next` to continue with `TreeMap` — the final structure in Module 9 before we move to Module 10 (Generics).

next

MODULE 9 (continued): TreeMap

Step 3: Syntax

java

```
import java.util.TreeMap;

public class Main {
    public static void main(String[] args) {
        TreeMap<String, Integer> scores = new TreeMap<>();

        scores.put("Charlie", 85);
        scores.put("Alice", 92);
        scores.put("Bob", 78);

        System.out.println(scores);
        System.out.println(scores.firstKey());
        System.out.println(scores.lastKey());

        for (Map.Entry<String, Integer> entry : scores.entrySet()) {
```

```

        System.out.println(entry.getKey() + " -> " + entry.getValue());
    }
}

```

Output:

```

{Alice=92, Bob=78, Charlie=85}
Alice
Charlie
Alice -> 92
Bob -> 78
Charlie -> 85

```

(Note: this requires `import java.util.Map;` as well, for the `Map.Entry` type used in the loop.)

Step 4: Breakdown

```

TreeMap<String, Integer> scores = new TreeMap<>();

```

Same two-generic-type pattern as `HashMap`. The behavioral difference: **keys are automatically sorted**.

```

scores.put("Charlie", 85); ... etc.

```

Same `.put()` method as `HashMap` — but notice the output: `{Alice=92, Bob=78, Charlie=85}` — sorted **alphabetically by key**, even though we inserted `Charlie` first.

```

scores.firstKey() / scores.lastKey()

```

Symbol	Meaning
<code>.firstKey()</code>	Returns the smallest key (alphabetically first, here).
<code>.lastKey()</code>	Returns the largest key (alphabetically last, here).

(These mirror `TreeSet`'s `.first()` / `.last()`, just applied to keys instead of plain elements.)

Step 5: Keyword Explanation

`TreeMap`

- **What it means:** A `Map` implementation that keeps its **keys** sorted at all times, using the same Red-Black Tree structure you learned about in `TreeSet`.
- **Why it exists:** Same motivation as `TreeSet` vs `HashSet` — sometimes you need key-value lookup **and** sorted iteration order, simultaneously. Examples: a leaderboard mapping player names to scores where you want to display them alphabetically, a price list that needs to stay sorted by product ID, a calendar mapping dates to events where chronological order matters.
- **When to use it:** Whenever sorted key order matters for your use case. If it doesn't matter, `HashMap` is simpler and generally faster.

`.firstKey()` / `.lastKey()`

- **What they mean:** Direct, fast ($O(\log n)$) access to the minimum/maximum key, found by walking down the appropriate side of the underlying tree.

Step 6: `TreeMap` Also Has Navigation Methods

Just like `TreeSet` had `.higher()` / `.lower()` / `.ceiling()` / `.floor()`, `TreeMap` has equivalents that operate on **keys**:

```
java
System.out.println(scores.higherKey("Bob")); // Charlie
System.out.println(scores.lowerKey("Charlie")); // Bob
System.out.println(scores.ceilingKey("Bob")); // Bob (inclusive)
System.out.println(scores.floorKey("Bob")); // Bob (inclusive)
```

These follow the exact same logic as before, just renamed with "Key" since we're now operating on a `Map` rather than a plain `Set`.

Step 7: How Does `TreeMap` Know the Sort Order? — `Comparable` Again

Exactly the same rule as `TreeSet`. `TreeMap<String, Integer>` works automatically because `String` implements `Comparable` (it knows how to alphabetically order itself). If your **key** type doesn't implement `Comparable`, you must supply a `Comparator`:

```
java
import java.util.TreeMap;
import java.util.Comparator;

TreeMap<String, Integer> scores = new TreeMap<>(Comparator.reverseOrder());
scores.put("Charlie", 85);
```

```
scores.put("Alice", 92);
scores.put("Bob", 78);
System.out.println(scores); // {Charlie=85, Bob=78, Alice=92} - reverse alphabet
```

Important distinction to internalize now: sorting always applies to the **key**, never directly to the value. `TreeMap` doesn't know or care how to sort `Integer` scores — it only sorts based on `String` names in this example, because `String` is the key type.

Step 8: Compiler Behavior

- Both generic types are enforced, same as `HashMap`.
- As with `TreeSet`, if the key type doesn't implement `Comparable` and you don't supply a `Comparator`, this compiles fine but throws a `ClassCastException` at **runtime**, the moment a second differently-keyed entry is inserted and a comparison is attempted.

Step 9: Runtime Behavior — Full Map Comparison

Operation	HashMap	TreeMap
<code>.put()</code>	O(1) average	O(log n)
<code>.get()</code>	O(1) average	O(log n)
Key order	None guaranteed	Sorted
Underlying structure	Hash table (buckets)	Red-Black Tree

Decision guide:

- Need the fastest possible lookup, don't care about order → `HashMap`.
- Need sorted key iteration, range queries, first/last key access → `TreeMap`.

Step 10: Memory Behavior

Identical reasoning to `TreeSet` vs `HashSet`: each entry in a `TreeMap` is stored as a tree node (with left/right child references and a color marker for balancing), which carries **more memory overhead per entry** than `HashMap`'s simpler bucket-based node structure.

Step 11: Common Mistakes

- 1. Using `TreeMap` when sorted order isn't actually needed
Unnecessary $O(\log n)$ overhead instead of `HashMap`'s $O(1)$ average — a common performance mistake when developers default to `TreeMap` "just in case."
- 2. Using a custom class as a key without `Comparable` / `Comparator`
Same `ClassCastException` risk as `TreeSet`.
- 3. Calling `.firstKey()` on an empty `TreeMap`

```
java
TreeMap<String, Integer> empty = new TreeMap<>();
empty.firstKey(); // throws NoSuchElementException at runtime!
```

- 4. Forgetting that sorting is based on keys, not values
A common beginner expectation is "sort this map by score" (the value) — but `TreeMap` cannot do this directly; it only sorts by key. To sort by value, you'd need a different approach entirely (e.g., converting entries to a `List` and sorting that list with a custom `Comparator` — a technique we'll cover properly once we reach the Stream API in Module 13).

Module 9 Complete — Full Collections Framework Summary

Structure	Category	Ordered?	Duplicates?	Key Feature
<code>ArrayList</code>	List	Insertion order	Yes	Fast random access by index
<code>LinkedList</code>	List/Queue/Deque	Insertion order	Yes	Fast insert/remove at ends
<code>Stack</code>	Stack (legacy)	LIFO	Yes	Last-in-first-out
<code>Queue</code> (via <code>LinkedList</code>)	Queue	FIFO	Yes	First-in-first-out
<code>PriorityQueue</code>	Queue	Priority order	Yes	Min/max always accessible fast
<code>HashSet</code>	Set	None	No	Fastest uniqueness checking
<code>TreeSet</code>	Set	Sorted	No	Sorted + unique